صفحه برای ترجمه‌ی آدرس‌های منطقی به آدرس‌های فیزیکی استفاده می‌کند. همین‌طور، توسط اعزام‌کننده‌ی پردازنده در زمان تخصیص پردازنده به پردازه جهت تعریف جدول صفحه‌ی سخت‌افزاری استفاده می‌شود. بنابراین، صفحه‌بندی زمان برگردان متن را افزایش می‌دهد.

۹-۳-۲ پشتیبانی سخت‌افزاری

چون جدول صفحات، ساختمان‌داده‌های پردازه محسوب می‌شوند، اشاره‌گری به جدول صفحه در کنار مقادیر ثبات‌های دیگر (مثل اشاره‌گر دستورالعمل) در بلاک کنترل هر پردازه ذخیره می‌شود. زمانی که زمان‌بند پردازنده یک پردازه را برای اجرا انتخاب می‌کند، بایستی ثبات‌های کاربر را دوباره بارگذاری کرده و مقادیر مناسب جدول صفحه‌ی سخت‌افزاری را از جدول صفحه‌ی ذخیره شده‌ی کاربر تعریف نماید.

پیاده‌سازی سخت‌افزاری جدول صفحه به روش‌های متعددی قابل انجام است. در ساده‌ترین حالت، جدول صفحه به صورت مجموعه‌ای از **ثبات‌های** سخت‌افزاری اختصاصی پیاده‌سازی می‌شود که ترجمه‌ی آدرس صفحه را بسیار مؤثر می‌سازد. با این وجود، این رویکرد، زمان برگردان متن را افزایش می‌دهد، چون هر یک از این ثبات‌ها بایستی در طی یک برگردان متن معاوضه شوند.

استفاده از ثبات‌ها برای جدول صفحه تا زمانی مطلوب است که جدول صفحه تا اندازه‌ای کوچک (برای مثال، ۲۵۶ واردهای) باشد. با این وجود، اکثر پردازنده‌های امروزی جداول بسیار بزرگی (برای مثال، 2^{20} واردهای) را نیز پشتیبانی می‌کنند. در این ماشین‌ها، پیاده‌سازی جدول صفحه با استفاده از ثبات‌ها امکان‌پذیر نیست. به جای آن، جدول صفحه در حافظه‌ی اصلی نگه‌داری می‌شود و یک **ثبات مبنای جدول صفحه**[1] (**PTBR**) به جدول صفحه اشاره می‌کند. تغییر جداول صفحه فقط مستلزم تغییر این ثبات است که زمان برگردان متن را به طور قابل ملاحظه‌ای کاهش می‌دهد.

۹-۳-۲-۱ بافر ترجمه در کنار

هرچند که ذخیره‌سازی جدول صفحه در حافظه‌ی اصلی می‌تواند به برگردان‌های متن سریع‌تری منجر شود، امکان دارد که زمان‌های دسترسی حافظه را نیز کندتر سازد. فرض کنید که می‌خواهیم مکان i را دسترسی کنیم. نخست بایستی با استفاده از مقدار PTBR با افست شماره صفحه‌ی i، ایندکسی به جدول صفحه بزنیم. این کار به یک دسترسی حافظه نیاز دارد. با این دسترسی، شماره‌ی قاب بدست می‌آید که با افست صفحه ترکیب می‌شود تا آدرس واقعی را تولید نماید. پس از آن می‌توانیم مکان مطلوب حافظه را دستیابی کنیم. این طرح برای دسترسی داده‌ها به ***دو*** بار دسترسی حافظه نیاز دارد (یک بار برای وارده‌ی جدول صفحه و یک بار هم برای داده‌های واقعی). بنابراین، سرعت دسترسی حافظه با ضریب ۲ کند می‌شود، دیرکردی که در بسیاری از شرایط قابل تحمل نیست.

1. Page-table base register

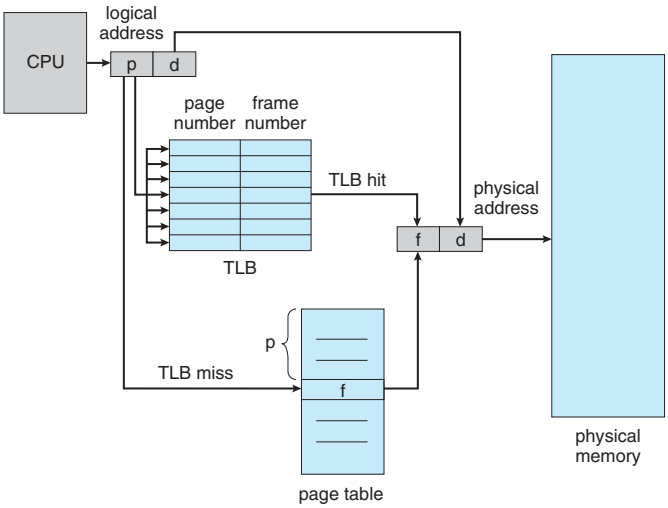
راه‌حل معمول این مسأله استفاده از یک کَش سخت‌افزاری خاص کوچک با امکان جستجوی سریع به نام **بافر ترجمه در کنار**[1] (**TLB**) است. TLB، حافظه‌ی انجمنی پرسرعت می‌باشد. هر یک از واردەهای TLB شامل دو قسمت است: یک کلید (یا tag) و یک مقدار. لحظه‌ای که یک عنصر به حافظه‌ی انجمنی عرضه می‌شود، این عنصر با تمامی کلیدها به صورت همزمان مقایسه می‌گردد. اگر عنصر پیدا شود، فیلد مقدار متناظر آن برگردانده می‌شود. این جستجو سریع است؛ رجوع به TLB در سخت‌افزار مدرن به عنوان بخشی از خطلوله‌ی دستورالعمل است که اصولاً هیچ‌گونه جریمه‌ی کارایی به بار نمی‌آورد. با این وجود، برای توانمندی اجرای جستجو در یک گام خطلوله بایستی TLB کوچک نگه داشته شود. نوعاً اندازه‌ی آن بین ۳۲ و ۱۰۲۴ وارده است. برخی از پردازنده‌ها TLBهای آدرس دستورالعمل و داده‌ها را جداگانه پیاده‌سازی می‌کنند. این کار می‌تواند تعداد واردەهای در دسترس TLB را دو برابر کند، چون مراجعه‌های آنها در گام‌های متفاوتی از خطلوله اتفاق می‌افتند. در این توسعه می‌توانیم نمونه‌ای از تکامل فناوری پردازنده را مشاهده کنیم: سیستم‌هایی که TLB نداشتند، به سیستم‌هایی تکامل یافتند که سطوح متعددی از TLBها را دارند، درست مانند آنهایی که سطوح متعددی از کَش‌ها دارند.

TLB با جداول صفحه به این صورت استفاده می‌شود. TLB فقط واردەهای اندکی از جدول صفحه را شامل است. موقعی که یک آدرس منطقی توسط پردازنده تولید می‌شود، MMU در ابتدا بررسی می‌کند که آیا شماره صفحه‌ی آن در TLB هست یا خیر. اگر شماره‌ی صفحه در آنجا بود، بلافاصله شماره‌ی قاب آن برای دسترسی حافظه استفاده می‌شود. همان‌گونه که عنوان شد، این گام‌ها به عنوان بخشی از خطلوله‌ی دستورالعمل در پردازنده اجرا می‌شوند که در مقایسه با سیستمی که صفحه‌بندی را پیاده‌سازی نمی‌کند، هیچ‌گونه جریمه‌ی کارایی به بار نمی‌آورد.

اگر شماره‌ی صفحه در TLB نباشد (معروف به **شکست**[2] **TLB**)، ترجمه‌ی آدرس با دنبال‌کردن گام‌های نشان داده شده در بخش ۹-۳-۱ به پیش می‌رود که بایستی مراجعه‌ای به حافظه جهت دستیابی جدول صفحه صورت گیرد. زمانی که شماره‌ی قاب بدست می‌آید، می‌توانیم از آن برای دسترسی حافظه استفاده کنیم (شکل ۹-۱۲). به علاوه، شماره‌ی صفحه و شماره‌ی قاب آن را به TLB اضافه می‌کنیم تا در مراجعات بعدی بتوان آن را به سرعت یافت.

اگر TLB قبلاً پر شده باشد، بایستی یک واردەی موجود برای جایگزینی انتخاب شود. سیاست‌های جایگزینی، از کمتر اخیراً به کار رفته[3] (LRU) با نوبت چرخشی گرفته تا تصادفی تغییر می‌کنند. برخی پردازنده‌ها به سیستم‌عامل اجازه می‌دهند تا در جایگزینی واردەی LRU مشارکت کند، در حالی که بقیه خودشان این موضوع را حل می‌کنند. به علاوه، در برخی TLBها امکان **میخکوبی**[4] واردەها نیز وجود دارد به این معنی که، نمی‌توان آنها را از TLB حذف کرد. نوعاً، واردەهایی از TLB که به کد کلیدی هسته مربوط هستند، میخکوب می‌شوند.

1. Translation look-aside buffer
2. TLB miss
3. Least recently used
4. Wired down



شکل ۹-۱۲ سخت‌افزار صفحه‌بندی با TLB.

برخی TLBها، **شناسه‌ی فضای آدرس** (**ASID**) را در هر یک از وارده‌های TLB ذخیره می‌کنند. هر ASID یک پردازه را به صورت یکتا مشخص می‌کند و برای فراهم‌ساختن حفاظت فضای آدرس آن پردازه استفاده می‌شود. لحظه‌ای که TLB می‌خواهد شماره‌ی صفحات مجازی را بدست آورد، بایستی مطمئن شود که ASIDی پردازه‌ی در حال اجرا با ASIDی مربوط به صفحه‌ی مجازی مطابقت دارد. اگر ASIDها با هم تطبیق نداشته باشند، این کار به عنوان یک شکست TLB محسوب می‌شود. علاوه بر امکان حفاظت فضای آدرس با استفاده از ASID، امکان وجود همزمان وارده‌های متعدد پردازه‌های متفاوت در TLB فراهم می‌شود. اگر TLB برای ASIDهای جداگانه پشتیبانی نداشته باشد، در این صورت در هر بار که جدول صفحه‌ی جدیدی انتخاب می‌شود، بایستی TLB به منظور اطمینان از این که پردازه‌ای که بعداً اجرا می‌شود از اطلاعات ترجمه‌ی اشتباه استفاده نخواهد کرد، **خالی شود** (یا پاک شود). در غیر این صورت، امکان دارد که در TLB وارده‌هایی باشند که هرچند آدرس‌های مجازی معتبری هستند، اما آدرس‌های فیزیکی نادرست یا نامعتبری دارند که از پردازه‌ی قبلی مانده‌اند.

درصد مواردی که شماره‌ی صفحه‌ی مطلوبی در TLB یافت می‌شود، **ضریب توفیق**[1] نام دارد. ضریب توفیق ۸۰ درصدی به این معناست که، ۸۰ درصد مواقع، شماره‌ی صفحه‌ی مطلوب خود را در TLB می‌یابیم. اگر دسترسی به حافظه ۱۰ نانوثانیه طول بکشد، در این صورت موقعی که شماره‌ی صفحه در TLB وجود دارد، دسترسی حافظه‌ی نگاشته شده ۱۰ نانوثانیه زمان می‌برد. اگر در یافتن شماره‌ی صفحه در TLB با شکست مواجه شویم، در این صورت بایستی ابتدا حافظه را برای جدول صفحه و شماره‌ی قاب دستیابی کنیم (۱۰ نانوثانیه) و سپس، بایت مطلوب را در حافظه بدست آوریم (۱۰ نانوثانیه)، که در مجموع ۲۰ نانوثانیه خواهد شد. (فرض ما بر این است که مراجعه به یک جدول صفحه فقط یک دسترسی حافظه لازم دارد، اما می‌تواند بیش

1. Hit ratio

از این باشد، چنان که خواهیم دید.) برای یافتن **زمان دسترسی مؤثر حافظه**[1]، وزن احتمالی هر یک از حالت‌ها را اعمال می‌کنیم:

زمان دسترسی مؤثر = ۰/۸۰ × ۱۰ + ۰/۲۰ × ۲۰

= ۱۲ نانوثانیه

در این مثال، با کندی ۲۰ درصدی زمان دسترسی حافظه مواجه هستیم (از ۱۰ به ۱۲ نانوثانیه). با ضریب توفیق ۹۹ درصدی، که بسیار واقع‌بینانه است، زمان دسترسی مؤثر را به صورت زیر بدست می‌آوریم:

زمان دسترسی مؤثر = ۰/۹۹ × ۱۰ + ۰/۰۱ × ۲۰

= ۱۰/۱ نانوثانیه

با این افزایش نرخ توفیق، دسترسی حافظه فقط ۱ درصد با کندی همراه است.

همان‌گونه که پیشتر متذکر شدیم، امروزه ممکن است پردازنده‌ها سطوح متعددی از TLBها را فراهم کنند. از این رو، محاسبه‌ی زمان‌های دسترسی حافظه در پردازنده‌های مدرن، خیلی پیچیده‌تر از آن خواهد بود که در مثال بالا نشان داده شد. برای نمونه، پردازنده‌ی Intel Core i7 دارای یک TLB دستورالعمل L1 با ۱۲۸ وارده و یک TLB داده‌ی L1 با ۶۴ وارده است. در صورت مواجهه با یک شکست در L1، برای پردازنده شش دور زمان می‌برد تا وارده‌ای را در TLBی L2 با ۵۱۲ وارده بررسی کند. یک شکست در L2 به این معنی است که پردازنده یا باید برای یافتن آدرس قاب مربوطه به سراغ واردەهای جدول صفحه در حافظه برود که می‌تواند صدها دور زمان بگیرد، و یا به سیستم‌عامل وقفه بدهد که این کار را به انجام برساند.

تحلیل کارایی کاملی از سربار صفحه‌بندی چنین سیستمی مستلزم اطلاعات نرخ شکست هر ردیف TLB می‌باشد. با این وجود، می‌توانیم از اطلاعات کلی بالا ببینیم که امکانات سخت‌افزاری می‌توانند تأثیر به سزایی در کارایی حافظه داشته باشند و این که، پیشرفت‌های سیستم‌عامل (همچون صفحه‌بندی) هم به نوبت خود می‌تواند از تغییرات سخت‌افزاری متأثر شود (همچون TLBها). پیامد ضریب توفیق بر TLB را با جزئیات بیشتری در فصل ۱۰ بررسی خواهیم کرد.

TLBها یک امکان سخت‌افزری بوده و از این رو به نظر می‌رسد که زیاد مورد توجه سیستم‌عامل‌ها و طراحان آنها قرار نگیرند. اما طراح نیاز دارد که کارکرد و امکانات TLBها را که با بستر سخت‌افزار تغییر می‌کنند، درک کند. برای عملیات بهینه، طراحی سیستم‌عامل برای یک بستر مشخص بایستی صفحه‌بندی را مطابق با طراحی TLB موجود پیاده‌سازی کند. همین‌طور، یک تغییر در طراحی TLB (برای مثال، بین نسل‌های متفاوت پردازنده‌های اینتل) ممکن است مستلزم تغییری در پیاده‌سازی صفحه‌بندی سیستم‌عامل‌هایی باشد که از آن استفاده می‌کنند.

1. Effective memory-access time

۹-۳-۳ حفاظت

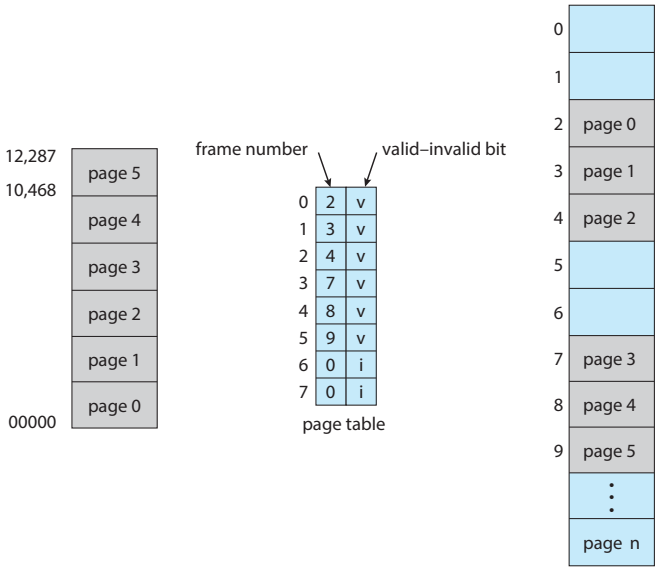
حفاظت حافظه در یک محیط صفحه‌بندی با لحاظ بیت‌های حفاظتی برای هر یک از قاب‌ها انجام می‌گیرد. این بیت‌ها علی‌القاعده در جدول صفحه نگه‌داری می‌شوند.

با یک بیت می‌توان یک صفحه را خواندنی-نوشتنی یا فقط‌خواندنی تعریف کرد. هر مراجعه به حافظه بایستی برای یافتن شماره‌ی قاب درست از میان جدول صفحه عبور کند. در همان حال که آدرس فیزیکی محاسبه می‌شود، می‌توان بیت‌های حفاظت را بررسی کرد تا تصدیق شود هرگز صفحه‌ای که فقط‌خواندنی است نوشته نمی‌شود. هر اقدامی که برای نوشتن یک صفحه‌ی فقط‌خواندنی صورت گیرد یک تله‌ی سخت‌افزاری به سیستم‌عامل (یا نقض حفاظت حافظه) را در پی خواهد داشت.

این رویکرد را به سادگی می‌توانیم تعمیم بدهیم تا سطح حفاظت ریزتری هم فراهم شود. می‌توانیم سخت‌افزاری برای داشتن حفاظت فقط‌خواندنی، خواندنی-نوشتنی یا فقط‌اجرایی ایجاد کنیم؛ یا با لحاظ بیت‌های حفاظت مجزا برای هر یک از انواع دسترسی‌ها می‌توانیم هر ترکیبی از این دسترسی‌ها را فراهم کنیم. هرگونه اقدام غیر قانونی در تله‌ی سیستم‌عامل خواهد افتاد.

بیت دیگری هم عموماً برای هر یک از واردهای جدول صفحه به نام بیت **معتبر-نامعتبر** متصل می‌شود. زمانی که این بیت *معتبر* انتخاب می‌شود، صفحه‌ی مربوطه در فضای آدرس منطقی پردازه قرار دارد و لذا، صفحه‌ای قانونی (یا معتبر) است. زمانی هم که *نامعتبر* انتخاب می‌شود، صفحه در فضای آدرس منطقی پردازه قرار ندارد. آدرس‌های غیر قانونی با استفاده از بیت معتبر-نامعتبر تله می‌شوند. سیستم‌عامل با انتخاب این بیت برای هر یک از صفحات، دسترسی یا منع دسترسی صفحه را فراهم می‌کند.

برای مثال، فرض کنید در سیستمی با فضای آدرس ۱۴بیتی (۰ تا ۱۶۳۸۳)، برنامه‌ای داریم که بایستی فقط از آدرس‌های ۰ تا ۱۰۴۶۸ استفاده نماید. با داشتن اندازه‌ی صفحات ۲ کیلوبایتی وضعیتی داریم که در شکل ۹-۱۳ نشان داده می‌شود. آدرس صفحات ۰، ۱، ۲، ۳، ۴ و ۵ مطابق معمول از طریق جدول صفحه نگاشته می‌شوند. با این وجود، هرگونه اقدامی که برای تولید آدرس صفحات ۶ یا ۷ صورت بگیرد، بیت معتبر-نامعتبر را نامعتبر یافته و کامپیوتر در تله‌ی سیستم‌عامل قرار خواهد گرفت (رجوع به صفحه‌ی نامعتبر).



شکل ۹-۱۳ بیت معتبر (v) یا نامعتبر (i) در یک جدول صفحه.

توجه دارید که این طرح باز هم مسأله‌دار شده است. چون برنامه فقط تا آدرس ۱۰۴۶۸ گسترش دارد، هرگونه مراجعه‌ای که فراتر از این آدرس باشد غیر قانونی است. با این وجود، مراجعات صفحه‌ی ۵ در رده‌ی معتبر قرار گرفته‌اند، لذا دسترسی آدرس‌ها تا ۱۲۲۸۷ معتبر می‌باشند. تنها آدرس‌های ۱۲۲۸۸ تا ۱۶۳۸۳ نامعتبر هستند. این مسأله به خاطر اندازه‌ی صفحات ۲ کیلوبایتی است و پارگی داخلی صفحه‌بندی را گوشزد می‌کند.

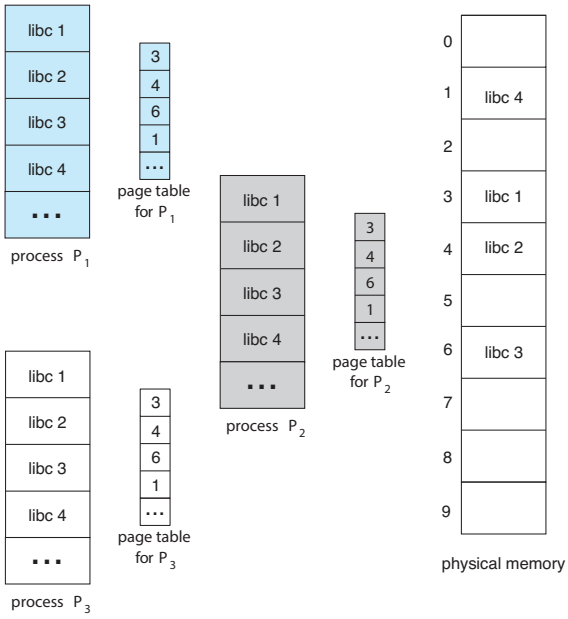
به ندرت اتفاق می‌افتد که یک پردازه از تمامی بازه‌ی آدرس‌های خود استفاده نماید. بسیاری از پردازه‌ها به واقع تنها کسر کوچکی از فضای آدرس در اختیار خود را استفاده می‌کنند. در چنین مواردی، ایجاد واردهای جدول صفحه برای هر یک از صفحات بازه‌ی آدرس‌های پردازه به نوعی اتلاف حافظه را در پی دارد. بیشتر واردهای این جدول بلااستفاده خواهند بود، اما فضای با ارزش حافظه را اشغال خواهند کرد. برخی سیستم‌ها، سخت‌افزاری به شکل **ثبات طول جدول صفحه** (**PTLR**) فراهم می‌کنند که اندازه‌ی جدول صفحه را نشان می‌دهد. مقدار این ثبات با هر یک از آدرس‌های منطقی مقایسه می‌شود تا تصدیق شود که آدرس تولید شده در بازه‌ی آدرس‌های معتبر پردازه قرار دارد. شکست در این آزمون، یک خطای تله به سیستم‌عامل را در پی دارد.

۹-۳-۴ صفحات مشترک

یک مزیت صفحه‌بندی در امکان *اشتراک* کد مشترک است، موضوعی که مخصوصاً در یک محیط با پردازه‌های متعدد اهمیت می‌یابد. کتابخانه‌ی استاندارد C را در نظر بگیرید که بخشی از رابط فراخوانی سیستمی نسخه‌های زیادی از یونیکس و لینوکس را فراهم می‌کند. در یک سیستم لینوکس نوعی، اکثر پردازه‌های کاربر به کتابخانه‌ی libc زبان C نیاز دارند. یک گزینه این است که برای هر پردازه، کپی خاص خود پردازه از libc را در فضای آدرس آن داشته

باشیم. اگر سیستمی ۴۰ پردازه‌ی کاربری داشته باشد و کتابخانه‌ی libc برابر ۲ مگابایت باشد، این کار نیازمند ۸۰ مگابایت حافظه است.

با این وجود، اگر کد، **کد چندمدخلی**[1] باشد، می‌تواند مطابق آنچه که در شکل ۹-۱۴ نشان داده می‌شود، به اشتراک گذاشته شود. در اینجا، سه پردازه می‌بینیم که صفحات کتابخانه‌ی libc زبان C را به اشتراک می‌گذارند. (هرچند که شکل نشان می‌دهد که libc چهار صفحه اشغال می‌کند، فضای بیشتری را در واقعیت اشغال می‌نماید.) کد چندمدخلی، کد غیرخودتغییر[2] است؛ یعنی، در طول اجرا هرگز تغییر نمی‌کند. بنابراین، دو یا چند پردازه می‌توانند کد یکسانی را همزمان اجرا نمایند. هر پردازه، کپی ثبات‌ها و حافظه‌ی داده‌های خاص خود را دارد که داده‌های حاصل از اجرای پردازه را نگه می‌دارند. البته، داده‌های دو پردازه‌ی متفاوت می‌توانند با هم فرق داشته باشند. تنها نیاز است که یک کپی از کتابخانه‌ی استاندارد C در حافظه‌ی فیزیکی نگه داشته شود، و جدول صفحه‌ی هر پردازه‌ی کاربر به کپی فیزیکی یکسانی از libc نگاشته می‌شود. لذا، برای پشتیبانی ۴۰ پردازه، تنها به یک کپی از کتابخانه نیاز داریم و اکنون، کل فضای لازم، ۲ مگابایت به‌جای ۸۰ مگابایت است—یک صرفه‌جویی قابل توجه!



شکل ۹-۱۴به اشتراک‌گذاری کتابخانه‌ی استاندارد C در یک محیط صفحه‌بندی.

علاوه بر کتابخانه‌های زمان اجرا همچون libc، سایر برنامه‌هایی که به‌صورت سنگین استفاده می‌شوند نیز می‌توانند به‌اشتراک گذاشته شوند—کامپایلرها، سیستم‌های پنجره‌ای، سیستم‌های پایگاه‌داده، و الی‌آخر. کتابخانه‌های مشترک بحث شده در بخش ۹-۱-۵ نوعاً با

1. Reentrant code
2. Non-self-modifying

صفحات مشترک پیاده‌سازی می‌شوند. برای این که امکان اشتراک وجود داشته باشد بایستی کد آن چندمدخلی باشد. نباید طبیعت فقط‌خواندنی کد مشترک به حساب صحت کد گذاشته شود؛ بایستی سیستم‌عامل این ویژگی را اجبار کند.

اشتراک حافظه مابین پردازه‌های یک سیستم مشابه اشتراک فضای آدرس یک وظیفه توسط نخ‌های آن است که در فصل ۴ عنوان شد. به علاوه، از فصل ۳ به خاطر دارید که اشتراک حافظه را به عنوان روشی برای ارتباط میان پردازه‌ها مطرح کردیم. برخی سیستم‌عامل‌ها، اشتراک حافظه را با استفاده از صفحات مشترک پیاده‌سازی می‌کنند.

سازماندهی حافظه مطابق صفحات، علاوه بر امکان اشتراک صفحات فیزیکی یکسان میان چندین پردازه، منافع متعدد دیگری هم دارد. چندین نفع دیگر را در فصل ۱۰ پوشش می‌دهیم.

۹-۴ ساختار جدول صفحه

در این بخش، برخی تکنیک‌های رایج ساختار جدول صفحه، اعم از صفحه‌بندی سلسله‌مراتبی، جدول صفحات درهم و جدول صفحات وارونه[1] را مرور می‌کنیم.

۹-۴-۱ صفحه‌بندی سلسله‌مراتبی

بیشتر سیستم‌های کامپیوتری مدرن، فضای آدرس منطقی بزرگی را پشتیبانی می‌کنند (2^{32} تا 2^{64} بایت). در چنین محیطی خود جدول صفحه می‌تواند بسیار بزرگ باشد. برای مثال، سیستمی با فضای آدرس منطقی ۳۲ بیتی را در نظر بگیرید. اگر اندازه‌ی صفحه در چنین سیستمی ۴ کیلوبایت (2^{12} بایت) باشد، در این صورت جدول صفحه می‌تواند بالای ۱ میلیون وارده ($2^{32}/2^{12} = 2^{20}$) داشته باشد. با فرض این که هر وارده ۴ بایت است؛ جدول صفحه به تنهایی، ۴ مگابایت از فضای آدرس فیزیکی هر پردازه را خواهد گرفت. روشن است که نمی‌خواهیم جدول صفحه را در حافظه‌ی اصلی به صورت پیوسته نگه بداریم. یک راه‌حل ساده برای این مسأله، تقسیم جدول صفحه به تکه‌های کوچک‌تر است. این تقسیم‌بندی را می‌توانیم به روش‌های متعددی انجام دهیم.

یک روش، استفاده از الگوریتم صفحه‌بندی دوسطحی است که در آن، خود جدول صفحه نیز صفحه‌بندی می‌شود (شکل ۹-۱۵). برای مثال، سیستم ۳۲ بیتی را باز هم با اندازه‌ی صفحات ۴ کیلوبایتی در نظر بگیرید. هر آدرس منطقی به یک شماره‌ی صفحه‌ی ۲۰ بیتی و افست صفحه‌ی ۱۲ بیتی تقسیم می‌شود. چون جدول صفحه را دوباره صفحه‌بندی می‌کنیم، شماره‌ی صفحه خود به یک شماره‌ی صفحه‌ی ۱۰ بیتی و یک افست صفحه‌ی ۱۰ بیتی تقسیم می‌شود. بنابراین، آدرس منطقی به صورت زیر خواهد بود:

شماره‌ی صفحه		افست صفحه
p_1	p_2	d
۱۰	۱۰	۱۲

1. Inverted page tables